Two models on one Lean file: what solved the problems, and what did not

Guang (Sunny) Yang

5 September 2026

The question

The submission (`submission/board_agent.py`, described in `docs/APPROACH.md`) puts `qwen/qwen3.5-flash-02-23` and `openai/gpt-oss-120b` on one shared Lean file. Each open `sorry` is a goal, a model takes a goal and writes one step, Lean judges it, and every statement a model writes is audited (by evaluation where the statement's binders allow it, by sampling for statements about a sequence, by the other model otherwise). This note asks what the pair solved that a single model in the kit's loop did not, and, run by run, whether the work was done by a model step, by the other model's check, or by a mechanism of the harness that asks no model at all.

Arms

Arm	What it is	Runs
qwen solo	the kit's `baselines/simple_agent.py`, unmodified, with `BASELINE_MODEL=qwen/qwen3.5-flash-02-23` (1200 s per problem, cut at 1080 s, 4 workers), the runs kept under `outputs/baseline/`	1 per problem
gpt-oss solo	the same baseline with `openai/gpt-oss-120b` (1200 s, cut at 1080 s, 5 workers), runs kept in the same place	1 per problem
board (pair)	the submission at commit `7a036d9`, `VM_TIME_LIMIT_S=1800` (sample problems) or `3600` (the harder six), `VM_BUDGET_USD=1.00`, one worker, on a 4-core machine	1 per problem, plus the other runs stated per row

The solo arms are the kit's draft-compile-repair loop, not the board with one model removed, and their clock was shorter, so the comparison says what the pair did that the shipped single-model loops did not. It does not isolate the second model as the cause. Where a solo arm was cut by the clock the table says `t/o` (the artifact's status is `cost_unknown`, the worker killed with a model call open).

Results

Score is the kit comparator's verdict. Wall time is the agent's own `wall_s` in seconds, comparator excluded. Cost is the OpenRouter spend of the board run, in dollars. "No leaves" is the same commit with `VM_LEAVES=off`, which skips the shape-built tactic blocks (`submission/leaves.py`) and leaves everything else in place, one run per problem (the `rmo_2001_2` run had not finished when this note was submitted). "Other runs" counts every other run of the same problem on the final commit and the seven commits before it, back to the one that introduced the leaf blocks, which differ from it only in those blocks and the three fixes to the search described in `docs/APPROACH.md`. A dagger marks a row whose run is on the commit before the final one, `12bf666`, the final commit's run of that problem not being the one kept under `outputs/board/` when this note was written.

Problem	qwen solo	gpt-oss solo	board	cost	no leaves	other runs
p01_linear	1 (65)	1 (60)	1 (4)	0	1 (4)	
p02_frac_cancel	1 (65)	1 (76)	1 (4)	0	1 (4)	
p03_sq_ge_two_ab	1 (69)	1 (99)	1 (48)	0.0005	1 (9)	
p04_sum_sq	1 (64)	1 (79)	1 (5)	0	1 (4)	
p05_gcd_mersenne	0 (363)	1 (140)	1 (4)	0	1 (4)	
p06_pow_mod	1 (135)	0 (713)	1 (77)	0	1 (77)	2 of 2 (63 s, 82 s)
p07_least_divisible	1 (391)	1 (795)	1 (30)	0.0009	1 (16)	
p08_sum_products	1 (70)	t/o	1 (39)	0.0005	1 (20)	
p09_imo1964	0 (819)	t/o	1 (60)	0.0004	1 (740)	1 of 1 (56 s)
p10_factorial_pow	0 (235)	t/o	1 (754)	0.041	1 (336)	5 of 5 (114 to 1220 s)
putnam_2018_a1	t/o	1*	1 (1854)	0.053	0 (2674)	8 of 9 (283 to 2611 s)
putnam_2020_a2	0 (614)	1*	1 (67)	0	0 (1234)	4 of 4 (48 to 58 s)
rmo_2000_2	0 (835)	t/o	1 (32)	0	0 (2595)	4 of 4 (23 to 26 s)
rmo_2000_3	t/o	t/o	0†		0 of 4	
rmo_2000_6	0 (904)	t/o	1 (276)	0.006	1 (1823)	9 of 16
rmo_2001_2	t/o	t/o	1 (439)†	0.003	not run	8 of 9 (408 to 648 s)
Total of 16	**7**	**8**	**15**		**11/14**	

* These two passes predate the kit's PR #9 ("Inline the Putnam answers so circular solutions stop scoring"). They used the answer to prove the answer and would not score now, which makes the gpt-oss solo total 6 on today's problem set.

On `rmo_2000_3`, the challenge file as shipped does not build under its own imports in the comparator (`Finset.sum` and `Finset.Ico` on ℕ need `Mathlib.Algebra.BigOperators.Group.Finset.Basic` and `Mathlib.Order.Interval.Finset.Nat`), so no solution can score on it as published. On a local copy with the two imports added, a proof of the statement written by hand compiles in the harness image in 0.6 s (a bound on each block `[j², (j+1)²)`, the block decomposition of the sum, the assembly), and the agent's runs on that copy are 0 of 5. The transcripts agree on the cause. Both models decompose the sum by `√k` instead of by the blocks and drop the factor 3 from the block bound, and the harness has a block for proving the right bound but nothing that chooses the right decomposition. That is a route search the submission does not have.

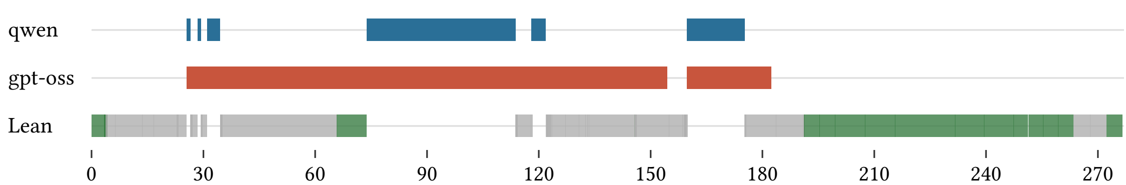
Seven of the sixteen other runs of `rmo_2000_6` failed. Five died on one goal, the lower bound `10 ≤ a * b` from the divisibility of `a ^ i * b ^ j` by 2000, before the `prime_to_bases` leaf existed, and two on faults in the cell mechanism that `docs/APPROACH.md` describes with the fixes. None bears on whether two models beat one.

What did the work

The transcripts of the passing runs record who wrote each accepted step and what closed a goal with no model asked. The table counts, for the run of each problem kept under `outputs/board/`, the accepted steps by model and the closures by the harness. The cocktail is the closing tactics tried before any model, the probes are the witness search, the harness's `#eval` of an answer slot and `apply?`, the leaves are the shape-built tactic blocks, and a recall is a statement proved earlier in the run and used again.

Problem	qwen	gpt-oss	cocktail	probes	leaf	conjecture	recall
p01_linear			1				
p02_frac_cancel			1				
p03_sq_ge_two_ab	1			1			
p04_sum_sq			1				
p05_gcd_mersenne			1				
p06_pow_mod			1	1			
p07_least_divisible			2				
p08_sum_products	1						
p09_imo1964			2		3		
p10_factorial_pow	11	3	4				
putnam_2018_a1	13	5	2		1		1
putnam_2020_a2				1	1	1	
rmo_2000_2					1		
rmo_2000_6	4	1	1	1	1		
rmo_2001_2	1	3			1		

Three layers, and on the harder problems the second is the one that scores. One of the hard runs end to end, to show where the clock goes:



seconds; model calls as bars (8 calls), Lean checks below (122, green accepted, grey rejected)

Lean was busy for 211 of the 276 s and the models for 216 s (one gpt-oss reply took 130 s), so the two overlap and neither alone is the bottleneck. The last 80 s are Lean checks that all pass, the closing sweep and the harness's own verification of the delivered file.

The first layer is the models' steps. Over the three hard runs with model steps (`rmo_2000_6`, `putnam_2018_a1`, `rmo_2001_2`) qwen wrote 18 of the 27 accepted steps from 37 tries and gpt-oss 9 from 19, and neither ordering holds on every run (on `rmo_2001_2` gpt-oss wrote more of the accepted steps, on `putnam_2018_a1` qwen's rate was the higher). gpt-oss does every model audit (2, 30 and 0 on those runs). The seats were fixed this way after 12 audits by qwen in which it named values that violated a hypothesis every time.

The second layer is Lean asked instead of a model. The cocktail closes p01, p02, p04 and p05 on the first check, the harness's `#eval` fills p06′s answer slot after both models failed to write it, and the leaf column is the tactic blocks built from the goal's shape and tried before a model, `pow_cycle` (the residues of `2 ^ n` modulo 7 on p09), `divisor_cases` from a product equation (`putnam_2018_a1`, `rmo_2001_2`), `prime_to_bases` from a divisibility by a numeral (`rmo_2000_6`), the two-sided squeeze of a cube (`rmo_2000_2`), `sum_induct` for sum identities. Each was written after a measured failure on a goal that both models had tried and missed, and each is dispatched on the goal's shape. Whether they carry to problems they were not written against is not measured in this note, and it is the question the holdout asks. On `rmo_2000_2` and `putnam_2020_a2` the whole proof is this layer and the run costs nothing.

With the leaves off (the "no leaves" column of the results table) the ten sample problems still pass, p09 in 740 s instead of 60 (the models find the cycle of `2 ^ n` modulo 7 by themselves, slowly). Of the hard problems, `rmo_2000_6` still passes, in 1823 s instead of 276 and for $0.066 instead of $0.006, and `putnam_2020_a2`, `rmo_2000_2` and `putnam_2018_a1` fail at the clock (80, 206 and 211 model calls). So the leaves decide the score on three of the five hard passes and the time on the other two, and none of the sample problems. That measures their share on these problems, not whether they carry to others.

The third layer is a generalisation the models could not produce. `putnam_2020_a2` (`∑_{j ≤ k} 2^(k−j) C(k+j, j) = 4^k`) is not closed under induction on `k`. Asked for a generalisation with one extra parameter, the two models gave 0 true and new statements in 32 samples (gpt-oss returned the same false family eight times). The harness splits the occurrences of `k` on the left into two groups, tabulates each family in Lean, fits a library of shapes to the table, verifies a fit below 11 in both variables and posts it as a lemma (`∑_{j ≤ k} 2^(k−j) C(n+j, j) = ∑_{i ≤ k} C(n+k+1, i)`). `sum_induct` proves the lemma in 0.5 s, the theorem is rewritten by it, and `apply?` returns `Nat.sum_range_choose_halfway`. No model wrote a step of that proof.

Where the pair added something, and where it did not

Over the runs kept from one day (v7.63 to v7.74, 59 runs), audit calls were about half of all calls and their reply latency about equal to the steps' (1990 s against 1921 s on one run, one audit reply holding the board lock for 482 s). What the audits decided, by who decided them, on the pair arm of the matrix below (6 runs) and on the 15 runs under `outputs/board/`:

Verifier	holds	refuted	unverified	not stated
evaluation walk, matrix pair arm	82	4	0	
gpt-oss, matrix pair arm	19	2	23	51
gpt-oss, final runs	22	4	16	14

The evaluation walk ($\mathbb{N}$ binders instantiated and the claim decided in Lean) refutes more than the auditing model does and never answers "unverified". Since the matrix the auditor is asked only what the walk cannot decide, which is why the final runs show no walk audits and 0 to 30 model audits per run. The sampled audit over a sequence (`rmo_2000_3`) refuted 3 of 47 model statements that the auditing model had passed as "unverified".

Where the second model did not help is the step itself. The matrix on v7.91 (1800 s, two runs per cell, the 24 runs kept under `outputs/matrix/`):

Arm	p09_imo1964	p10_factorial_pow	rmo_2000_6
pair, audit on (the submission)	1 of 2	2 of 2	2 of 2
qwen in both seats	1 of 2	2 of 2	0 of 2
pair, audit off	1 of 2	1 of 2	1 of 2
gpt-oss in both seats	0 of 2	0 of 2	0 of 2

gpt-oss alone is throughput, not knowledge (about 10 calls in 1240 s, each reply near the 4000-token cap at 12 to 27 tokens per second). The pair is the arm that did not lose `rmo_2000_6` in that matrix. Two runs per cell separate nothing finer than that.

Architecture

The single change that made the harder problems reachable was not a model setting but the unit Lean checks. One theorem is one heartbeat budget and one re-elaboration, so a leaf that needs 170000 heartbeats by itself never ran on a board with earlier steps (`rmo_2001_2`, five tries, every one at the limit). The file is now checked as cells, one goal one declaration (`docs/CELLS.md`), and delivered in the same shape. Schematically, what the models see on the left and what Lean checks and the comparator compiles on the right:

```
theorem rmo_2000_6 : A ∧ B := by
  refine ⟨?_, ?_⟩
  · refine ⟨?_, ?_⟩
    · exact ⟨5, 2, by norm_num⟩
    · intro n ⟨a, b, ha, hb, h, rfl⟩
      -- cell 7
      prime_to_bases 2 h; …
  · sorry
```

```
theorem vm_cell_7 (a b : ℕ) (ha : 0 < a)
     (hb : 0 < b) (h : 2000 ∣ a ^ 2 * b ^ 5) :
     10 ≤ a * b := by
    prime_to_bases 2 h; prime_to_bases 5 h; …

    theorem rmo_2000_6 : A ∧ B := by
      refine ⟨?_, ?_⟩
     · refine ⟨?_, ?_⟩
       · exact ⟨5, 2, by norm_num⟩
       · intro n ⟨a, b, ha, hb, h, rfl⟩
         exact vm_cell_7 a b ‹_› ‹_› ‹_›
     · sorry
```

What remains is a search structure. The board keeps three whole-file branches ranked by open-goal count, a stuck goal resets its cell or restarts its declaration, and a route that was 80% right is thrown away with the wrong 20% (one `rmo_2000_6` run reset the same cell 9 times with the same opening). Three of this week's fixes were to that structure rather than to any proof. A statement proved once is now recalled when the same goal reappears after a reset, a model that returns the same step for a goal is not offered that goal again until the board changes, and a hypothesis under an inaccessible name is a binder of its cell rather than a premise the link forgot to pass. `rmo_2000_3` is the problem this structure cannot reach. The pieces of its proof are in the harness, and nothing in the harness chooses between two decompositions of a sum. A proof tree with alternatives as siblings, so that a wrong decomposition is one branch and not the whole board, is the next change, and it was not made before this note.

What would settle it

Three experiments this note does not contain. The board on a problem no leaf was written against, which is what the holdout is. The kit's baseline at 3600 s and one worker on the six problems where it was cut by the clock, so that the solo arms and the board have the same budget. And the board with one model in both seats, and with the audit disabled, on the six harder problems rather than the three in the v7.91 matrix, so that the second model's contribution on the problems that need it is measured and not inferred.